

bulk-promote-test ワークフロー

データ生成パイプライン仕様書

本書は `.github/workflows/bulk-promote-test.yml` が QID 入力から `data/manga/*.yaml` を生成するまでのフローを各ステップ単位で説明する。

対象ブランチ: `claude/manga-database-affiliate-3x0ms` 更新日: 2026-05-07

全体パイプライン

ワークフローは「QID 列を空白区切りで受け取り、SQLite にデータを順に積み上げてから draft YAML を吐く」1 ジョブ構成。ジョブは何度でも回せる (= 毎回 `db:init` で空の DB から作り直す)。

①	INPUT	qids / max_pages / include_adult
②	INIT	DB 空構築 + マスタ取込 (publishers / magazines / mangaka 6,751 名)
③	SEED	adult publisher / mangaka / imprint シード (false-positive/negative 補正)
④	NDL	QID ループで NDL Search SRU API → series / editions / volumes 投入
⑤	WIKI	series ループで Wikipedia → kana / magazine / publisher / synopsis / genres
⑥	OBD	openBD bulk → title_kana が NULL の series のみ補完
⑦	PROMOTE bulk	adult_score 計算 + draft YAML を <code>data/manga/_drafts/</code> に書出し
⑧	PROMOTE drafts	placeholder 0 件のみ <code>data/manga/</code> に昇格
⑨	ARTIFACT	SQLite + drafts + 確定 yaml を ZIP で artifact 化 (auto-commit はしない)

重要な性質 (= 仕様の前提):

- ・ 毎回 DB はリセット — 過去 run の影響は無く、入力 QID + master だけで完全再現可能。
- ・ auto-commit しない — 結果は artifact (ZIP) に格納されるのみ。ユーザが ZIP を取り出して、私が ローカルで commit する運用。
- ・ placeholder 防衛線が二重 — `promote:bulk` は placeholder 入りでも `_drafts/` に書く / `promote:drafts` は placeholder 0 件のみ `data/manga/` に昇格。結果として「本番に乗るのは Wikipedia 連携が当たった series のみ」。
- ・ `data/manga/*.yaml` は不可侵 — 既存ファイルは `promote:drafts` が `--force` 無しでスキップ。
ピュアな観察をしたい時はユーザ事前に削除。

Step ① 入力 (workflow_dispatch)

パラメータ	例	説明
qids	Q219948 Q193300	空白区切りの Wikidata QID。正規表現 <code>^Q[0-9]+(+Q[0-9]+)*\$</code> で前段検証。
max_pages	2	1 人あたりの NDL ページ数 (1 ページ = 最大 200 件、1~10)。
include_adult	false	true なら CSV 上 has_adult_credit=1 の作家も対象。既定 false。

代替: `data/seed/scaling-canonical-21.txt` 等の seed file の内容を `qids` 欄に貼ることで一括指定可能 (現在 20 QID)。

Step ② DB 初期化 + マスタ取込

Step 2-a	<code>npm run db:init</code>
目的	.cache/db.sqlite を空から再生成 (schema_version 6)。
処理	schema.sql で series / editions / volumes / mangaka / publishers / magazines / adult_publishers / adult_mangaka_known / adult_imprints / sources / adult_signals 等のテーブルを CREATE。
Step 2-b	<code>npm run db:import:masters</code>
目的	data/publishers.yml / data/magazines.yml / data/genres.yml / data/demographics.yml を DB に投入。
処理	各 YAML の key を主キーにし、後段で Wikipedia 由来テキストとの紐付け (=「ジャンプ」→ magazine_key=jump-shounen) に使用。
Step 2-c	<code>npm run db:import:mangaka</code>
目的	Wikidata から事前 export した mangaka CSV (6,751 名) を mangaka テーブルに INSERT。
処理	QID / 表記揺れ alt_names (CSV カンマ区切り) / has_adult_credit / primary 表記名 (= name 列) を持つ。後段の NDL fetch で QID から名前を解決する辞書として機能。

Step ③ Adult 検出シード

promote:bulk が adult_score を計算する前提となる「既知の adult publisher / mangaka / imprint」セットを 3 ソースから seed する。

Step 3-a	<code>npm run fetch:adult-lists</code>
目的	成人向け publisher / mangaka を Wikipedia から fetch して adult_publishers / adult_mangaka_known テーブルを埋める。
処理	ja.wikipedia.org の「成人向け漫画雑誌の一覧」「日本の成人向け漫画家の一覧」をパースし、publisher 名 (mainstream は DENY_LIST で除外) / mangaka 名 (NFKC 正規化、length 1~40 文字) を抽出。
近況の対策	Tier 1B (2026-05-06) で講談社・白泉社・集英社・小学館・KADOKAWA・芳文社等 21 社を DENY_LIST 追加 (= 大手 mainstream の false-positive 排除)。
Step 3-b	<code>npm run seed:adult-imprints</code>
目的	imprint レベルの adult シグナルを seed (= manga-db.com dump 由来 ~250 entry)。
処理	data/seeds/adult-imprints.yml を読んで adult_imprints テーブルへ INSERT。imprint 名 (= 茜新社の「COMIC LO」 / リイド社の「クリベロン」等) で series を判定可能にする (= publisher 単位より granular)。

Step ④ NDL Search → series / editions / volumes 投入

QID を 1 個ずつループし、国会図書館 NDL Search の SRU API (CQL) で当該作家の漫画を検索。見つかった record を baseTitle + author の組合せで series 単位に集約し、edition 種別 (= standard / new-edition / wide / bunko / etc.) を分類して volumes に展開する。

4-a. CQL クエリ構築 (per QID)

CSV から取得した primary name + alt_names を OR で並べる。2026-05-07 fix: alt_names のうち length < 3 文字のもの (=「きい」「ONE」等) は CQL から除外し、別作家の record 混入を防ぐ。

```
creator="諫山創" OR creator="進撃の巨人 諫山創" AND ndc="726"
```

4-b. record 走査時の二重フィルタ

upsertVolume() の冒頭で 2 つのガードが順に発火する:

(i) 著者一致検証 v2 (NDL bug fix v2, 2026-05-07): NDL の creators 配列を normalizeCreatorName 後、我々の "expected names" (primary + alt_names、length filter 無し) のいずれかと 完全一致 する場合だけ受け入れる。substring 一致は使わない (=「堀田きいち」が「きい」を含んでも別人物)。

```
[skip-mismatch] "春風のエトランゼ" — NDL creators=[紀伊カンナ] ≠ きい
```

(ii) NDLC=Y84 prefix フィルタ (novelization 排除, 2026-05-07): NDC=726 (漫画類) では「金色のガッシュベル!! 第1巻」by 矢高鈴央 (= 児童文学/小説, NDLC=Y16 だが NDC=726.1) のような小説版が混入する。ndlc が "Y84" 始まりでないものは skip。ndlc 不在時は NDC=726 通過を尊重。

```
[skip-ndlc] "金色のガッシュベル!! 第1巻" — NDLC=Y16 (= 漫画外、ノベライズ等)
```

4-c. 集約ロジック

- ・ baseTitle: dcndl:edition / title / seriesTitle / partTitle を順に試して edition suffix (=「新装版」「文庫版」等) を classifyEdition() でタグ付け。
- ・ seriesKey: baseTitle + author 識別子で連結 (= 同タイトル別作家を分離)。同じ key なら series 統合 (= 既存 series_id を再利用)。
- ・ edition: standard / new-edition / wide / bunko / kanzen / aizou / etc. の type 列で 1 series に複数 edition 並列。
- ・ volumes: ISBN13 を主キーに UPSERT。巻数 / 発売日 / cover_url (= 後段 openBD 由来) / is_extra フラグを持つ。

Step ⑤ Wikipedia 補完 (kana / publisher / magazine / synopsis / genres)

DB に積まれた series を 1 件ずつ Wikipedia で照合し、NDL では取れない情報を補完する。1.1 秒/req のレート制限、全 series 試行 (= --limit 撤去済)。

5-a. 記事発見 (Layer A)

REST summary API → Parse wikitext API の 2 段。disambiguation page は弾く。タイトル直接 match のみで disambiguation suffix '(漫画)' 等の試行はしない (= 既知の limitation)。

5-b. infobox 抽出 (Layer B)

Infobox animanga/Header / Print / Manga / Infobox 漫画 / Infobox 文学作品 を順に試行、先勝ち。fields: 作者 / 出版社 / 掲載誌 / ジャンル / 巻数 / 連載期間 / ふりがな / 等。

5-c. 著者一致検証 (今回の追加、2026-05-07)

infobox の 作者 / 漫画 / 作画 / 著者 いずれかから wikiAuthorRaw を抽出し、DB の series_authors → mangaka.name (= expected_authors) と normalizeCreatorName 後に 完全一致 or 双方向 substring で照合。不一致なら synopsis / kana / 等を記録せず url のみ setNotFound して skip。

これにより 矢高鈴央 4コマ「ガッシュベル」が 雷句誠 主作「金色のガッシュ!!」の Wikipedia 記事を pull する pollution を防ぐ。

5-d. master 解決 (Layer C)

infobox の publisher / magazine / genres 文字列を NFKC 正規化 + substring match で publishers.yml / magazines.yml / genres.yml の key に解決。解決失敗時は raw text を保存しない (= 後続の placeholder 化)。

Step ⑥ openBD bulk (kana のみ補完)

Wikipedia ヒット率 18% の天井対策。series.title_kana が NULL/空のものだけを対象に、代表 ISBN を openBD API に投げて collationkey (= ヨミガナ) を取得。publisher / synopsis / cover は probe で 1~2% カバーと判明し ROI 低、触らない方針。

既に Wikipedia 由来 kana が入っている series は skip (= overwrite しない)。

Step ⑦ promote:bulk → drafts 生成

7-a. adult_score 計算 (per series)

computeAdultScore() で 4 シグナルを合算 (max 重み):

シグナル	weight	条件
wikidata_hentai_credit	2	CSV の has_adult_credit=1
wikipedia_adult_mangaka_list	2	作者名が adult_mangaka_known に存在
adult_publisher_imprint	3	imprint 文字列が adult_publishers にマッチ
adult_imprint	3	imprint 文字列が adult_imprints にマッチ (publisher 単位より granular、排他)

score >= 3 なら series ごと skip (= include_adult=true 指定時を除く)。score 自体は 0 でも DB に書いて、post-promote 段階で db:report --adult で観察可能。

7-b. draft YAML 構築

series 1 件 = draft YAML 1 ファイル。各 field の埋め方:

field	第一候補	fallback
title	series.title (NDL 由来)	—
title_kana	series.title_kana (Wikipedia → openBD 順)	TODO_kana
title_romaji	baseSlug (= kana から romaji 化、collision suffix なし)	TODO_romaji
publisher	series.publisher_key (Wikipedia 由来)	imprint → publishers.yml 逆引き → TODO_publisher
magazine	series.magazine_key (Wikipedia 由来)	null
genres	series.genres (Wikipedia 由来)	[TODO_genre]
demographic	series.demographic (Wikipedia 由来)	"shounen"
synopsis	series.synopsis (Wikipedia summary 800 字)	空文字

year_started / ended	standard edition の min/max 発売年	series 全体の min/max
authors	series_authors の最優先 1 名 (writer_artist > writer > artist > original_author)	無ければ skip
editions / volumes	buildEditionsForSeries() で type 別に纏めた配列	空 series は skip (min_volumes 未満)
wikipedia_url	series.wikipedia_url (記事ヒット時)	無ければ field 自体省略

7-c. ファイル名 (slug) 生成

deriveBaseSlug(): title_kana → romaji 化 (kuroshiro)、hyphen 区切り、lower-case。既存 slug と衝突する場合は -2 / -3 suffix を applyCollisionSuffix() で付加。ただし title_romaji 値には suffix 無しの baseSlug を使う (= 「inuyasha 2」のような巻数誤認を防ぐ)。

Step ⑧ promote:drafts → 本番 yaml への昇格

data/manga/_drafts/*.yaml を走査し、placeholder が 0 件 のものだけ data/manga/slug.yaml に move する。
placeholder の検出条件:

title_kana == 'TODO_kana'
title_romaji == 'TODO_romaji'
publisher == 'TODO_publisher'
magazine == null
genres に 'TODO_genre' を含む
synopsis == ""

placeholder が 1 件でもあれば draft のまま据え置き (= _drafts/ から移動しない)。data/manga/slug.yaml が既に存在する場合 --force 無しでスキップ (= 既存ファイル保護)。昇格時には末尾で MangaSchema 検証 + loadAllManga 全体検証を通過したものだけ書く。

Step ⑨ Artifact 化

ZIP 名: bulk-test-run-<run_number> (`if: always()` で失敗時も保存)

含まれるパス:

- ・ .cache/db.sqlite — SQLite DB 全体
- ・ .cache/ndl/ — NDL 生 XML キャッシュ (per QID, per page)
- ・ .cache/wikipedia/ — wikitext 生キャッシュ (per series)
- ・ data/manga/_drafts/ — promote:bulk が出した全 draft
- ・ data/manga/*.yaml — promote:drafts が昇格した production yaml

workflow は git に push しない。ユーザが ZIP を取り出してチャットに渡し、私 (Claude) が ローカルで unzip → commit → push する流れ。理由: GH Actions bot commit は他 push-trigger workflow を発火しないため、deploy が走らなくなる + run-by-run の人間レビュー機会を確保。

付録: 走行中に出力される観察 step

- ・ DB summary report — series 件数 / adult スコア分布 / sources 行数の総当たり集計

- ・ DB top 30 series with edition breakdown — 巻数の多い順に上位 30 series を edition 別に表示 (= 重複/分割の検査)
- ・ DB top 30 (post-Wikipedia) — Wikipedia 連携後の同集計 (= metadata 充足度の差分)
- ・ DB adult report (post-promote) — adult_score / adult_signals の最終結果 (= どの series がどの signal で skip されたか)
- ・ SQL diagnostics — editions.imprint 分布 / magazine_key 充足率 / adult_imprints 件数 / imprint × adult_imprints 交差
- ・ Drafts quality stats — draft 全体の placeholder 数別分布 (0 = production-ready / 1~5 placeholder の件数集計)

既知の制約 / 今後の対処予定

- ・ NDL データに magazine_key は無い — Wikipedia 連携経由のみ (= 現状 ~0.4% 充足)。demographic 判定もここに依存するため、magazine が埋まらないと shounen で fallback。
- ・ Wikipedia ヒット率 ~18% — 残り 82% は title_kana / publisher / magazine / synopsis / genres が NDL のみで完結せず placeholder 化、promote:drafts で昇格しない。
- ・ openBD は kana 限定 — publisher/synopsis/cover の openBD probe は ROI 低と判明、採用見送り。
- ・ Amazon PA-API は Phase 5 待ち — schema (amazon_metadata 表) は予約済、承認後に巻単位の cover / 価格 / Editorial Reviews を取得予定。
- ・ NDL の dcndl:seriesTitle が無い古い記録 — baseTitle 自前削りで edition suffix を取り違える可能性。sample run では「夜王 vol0 ×12」等の duplicate vol 数が観察されている。

— 仕様書ここまで —